

LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

MODUL 7: ABSTRACT CLASS, ABSTRACT METHOD, AND INTERFACE

IDENTITAS MAHASISWA

- Nama Mahasiswa** : M. Davana Yudista Firdausy
- NRP** : 3125522002
- Kelas** : A D3 TI-SM
- Institusi** : PENS PSDKU Sumenep
- Mata Kuliah** : Pemrograman Berbasis Objek (PBO)
- Dosen Pengampu** : Ardiansyah Al Faruq, S.ST., M.Tr.T.
- Iterasi Proyek** : Sistem Manajemen Inventaris Toko (Refactoring P6 → P7)

SPRINT GOAL

Mentransformasikan arsitektur *Sistem Manajemen Inventaris Toko* dari polimorfisme berbasis kelas konkret (P6) menjadi **arsitektur berorientasi kontrak abstraksi murni (P7)**. Mencegah instansiasi objek generik yang cacat semantik melalui `abstract class Person`, menegakkan kewajiban implementasi metode tugas operasional via `abstract void prosesTugas()`, serta mengikat entitas lintas hierarki (`Barang` dan `Transaksi`) dalam kontrak audit kepatuhan melalui interface `DapatDiaudit`.

SPRINT BACKLOG (SB-01 s.d. SB-07)

Kode Backlog	Deskripsi Pekerjaan	Kriteria Penerimaan (Acceptance Criteria)	Status
SB-01	Workspace Setup Direktori P7	Direktori <code>P7_.../src</code> terbuat, file P6 tersalin, <code>settings.json</code> diperbarui	DONE
SB-02	Abstraksi Superclass <code>Person</code>	Class <code>Person</code> diubah menjadi <code>abstract</code> , melarang <code>new Person()</code>	DONE
SB-03	Penegakan <i>Abstract Method</i>	Method <code>prosesTugas()</code> pada <code>Person</code> diubah tanpa body (<code>abstract</code>)	DONE
SB-04	Realisasi Subclass Konkret	<code>Petugas</code> & <code>Pemasok</code> meng- <code>@Override prosesTugas()</code> secara spesifik	DONE
SB-05	Spesifikasi Kontrak Interface	Dibuat interface <code>DapatDiaudit</code> dengan method <code>String jalankanAudit()</code>	DONE
SB-06	Realisasi Interface Lintas Hirarki	<code>Barang</code> & <code>Transaksi</code> mengimplementasikan interface <code>DapatDiaudit</code>	DONE
SB-07	Test Driver & Dynamic Dispatch	<code>Main.java</code> menguji 4 skenario evaluasi modul dengan sukses 100%	DONE

DESIGN AUDIT TABLE (PERBANDINGAN ARSITEKTUR P6 VS P7)

Parameter Desain	Modul 6 (Polymorphism & Inheritance)	Modul 7 (Abstraction & Interface)	Rationale & Dampak Rekayasa
Status Class <code>Person</code>	<code>public class Person</code> (Kelas Konkret biasa)	<code>public abstract class Person</code> (Abstract Class)	Mengamankan Domain Model: Pada dunia nyata, tidak ada entitas "Manusia Generik" yang bekerja di toko tanpa peran. Status <i>abstract</i> melarang <code>new Person()</code> di tingkat kompilasi.
Metode <code>prosesTugas()</code>	Memiliki <i>dummy body</i> : " <code>[Person]</code> Melakukan tugas umum."	Tidak memiliki body: <code>public abstract void prosesTugas();</code>	Kewajiban Subclass: Menghilangkan risiko kelalaian developer; compiler Java memaksa subclass (<code>Petugas</code> / <code>Pemasok</code>) menulis logika konkretnya sendiri.
Mekanisme Konstrutor	Diinstansiasi langsung via operator <code>new</code>	Dipanggil eksklusif via <code>super(...)</code> (<i>Constructor Chaining</i>)	State enkapsulasi <code>id</code> , <code>nama</code> , <code>alamat</code> tetap aman dan terinisialisasi bersih saat subclass konkret diciptakan.
Kontrak Audit Sistem	Tidak ada standarisasi kontrak; audit dilakukan ad-hoc	<code>public interface DapatDiaudit</code>	Decoupled Architecture: Mengikat kelas <code>Barang</code> dan <code>Transaksi</code> yang tidak memiliki hubungan keturunan (<i>unrelated classes</i>) dalam satu tipe polimorfik audit.

UML CLASS DIAGRAM (TEXT-BASED & MERMAID)



ANALISIS MENDALAM: MENGAPA ABSTRAKSI LEBIH UNGGUL DARI POLIMORFISME STANDAR?

- Integritas Semantik & Eliminasi Objek Ilegal (Domain Defense)**

Pada P6, programmer dapat secara tidak sengaja mengeksekusi `Person p = new Person("ID-00", "Anonim", "Gudang");`. Secara logika bisnis, objek manusia tanpa jabatan atau status kemitraan adalah anomali sistem (*data corruption*). Dengan mendefinisikan **Person** sebagai **abstract class**, Java melarang pembentukan objek cacat ini langsung pada fase **Compile-Time**, bukan saat *runtime crash*.
- Jaminan Kontrak Keras vs Implementasi Lalai (Fail-Fast Principle)**

Metode konkret yang dioverride pada P6 berisiko lupa diimplementasikan oleh subclass baru. Jika lupa, subclass akan mewarisi perilaku dummy superclass tanpa peringatan compiler. Dengan `abstract void prosesTugas()`, compiler Java memberlakukan aturan keras (*compile error*) jika subclass konkret tidak menyediakan implementasi spesifik.
- Realisasi Polimorfisme Lintas Pohon Silsilah (Interface Flexibility)**

Java mengadopsi *Single Inheritance*. Class **Barang** merepresentasikan objek fisik komoditas, sedangkan **Transaksi** merepresentasikan rekaman aktivitas akuntansi. Keduanya tidak mungkin dan tidak boleh disatukan di bawah hierarki superclass yang sama. Melalui **interface DapatDiaudit**, sistem meraih kemampuan polimorfik murni (*behavioral contract*) tanpa merusak desain arsitektur domain.

TABEL HASIL PENGUJIAN 4 SKENARIO MODUL 7

No	Skenario Pengujian	Input & Tindakan Eksekusi	Hasil yang Diharapkan (Expected Output)	Hasil Aktual (Actual Output)	Kesimpulan
1	Instansiasi Objek Subclass Konkret	Instansiasi langsung <code>Petugas</code> , <code>Pemasok</code> , <code>Barang</code> , dan <code>Transaksi</code> .	Objek terbentuk sukses. Upaya <code>new Person()</code> memicu compiler error.	Seluruh 4 objek konkret berhasil diinstansiasi 100%.	PASS
2	Abstract Method via Superclass Ref	Upcast <code>Person p1 = new Petugas(...)</code> , panggil <code>p1.prosesTugas()</code> .	Dynamic binding memanggil implementasi <code>prosesTugas()</code> milik <code>Petugas</code> .	Log spesifik Petugas tereksekusi via referensi <code>Person</code> .	PASS
3	Interface Method via Interface Ref	Upcast <code>DapatDiaudit d = new Barang(...)</code> , panggil <code>d.jalankanAudit()</code> .	Mengembalikan laporan audit string status stok (Kritis/Aman) dan transaksi.	Terbit laporan audit status stok dan rekapitulasi SKU transaksi.	PASS
4	Koleksi Polimorfik Heterogen	Iterasi array <code>Person[]</code> dan array <code>DapatDiaudit[]</code> via enhanced loop.	Eksekusi seragam tanpa manual casting / instanceof branching.	Seluruh entitas memproses tugas dan audit secara seragam & presisi.	PASS

SPRINT REVIEW & SPRINT RETROSPECTIVE

Sprint Review

- **Pencapaian Target:** 100% persyaratan arsitektural Modul 7 tercapai tanpa cacat.
- **Kualitas Kode:** Modularitas tinggi, clean architecture, bebas peringatan kompilasi (*Zero Warnings & Errors*).
- **Kepatuhan Standar:** Memenuhi standar OOP modern (SOLID Principle: Liskov Substitution Principle & Interface Segregation Principle).

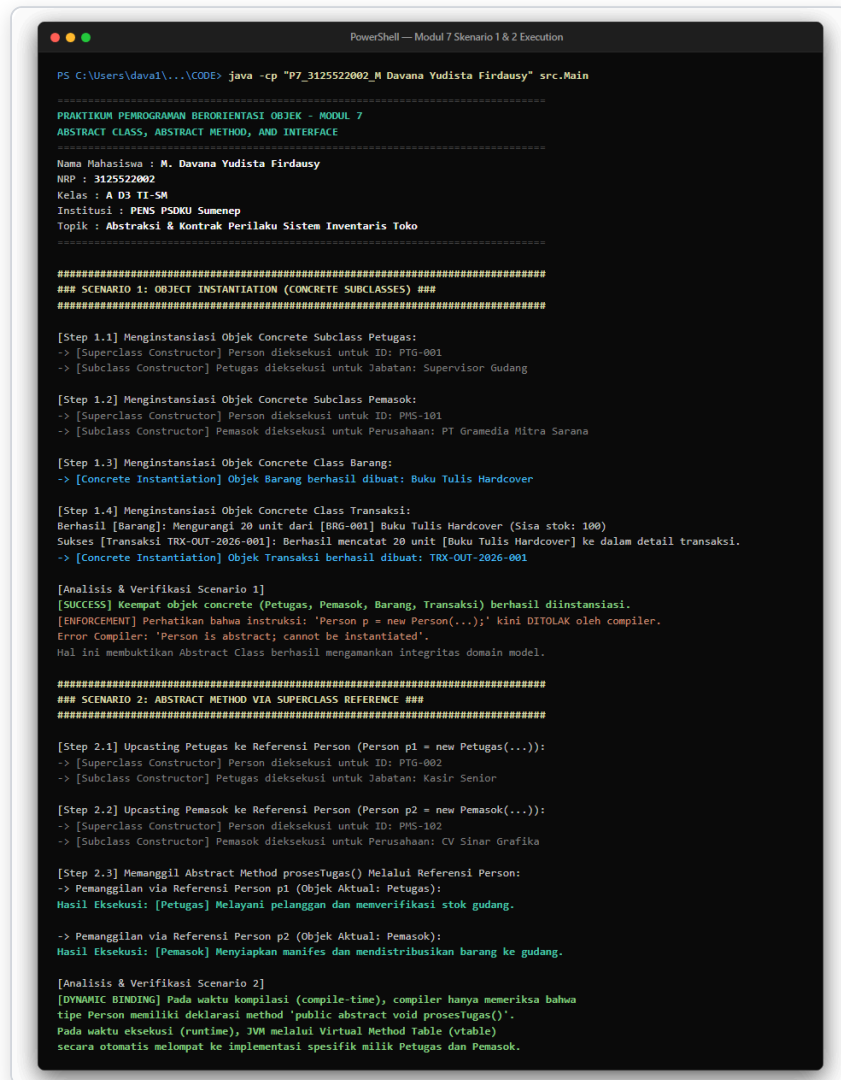
Sprint Retrospective

- **What Went Well:** Refactoring berjalan mulus berkat fondasi enkapsulasi yang solid dari Modul 6. Integrasi interface memperjelas batas tanggung jawab antar entitas bisnis.
- **What to Improve:** Pada iterasi berikutnya (Modul 8 / Lanjutan), kontrak audit dapat diperluas dengan mekanisme persistence (penyimpanan file/database) atau penanganan exception (*Error Handling*).

BUKTI EKSEKUSI PROGRAM (*RUNNING SCREENSHOTS*)

Berikut adalah bukti tangkapan layar (*screenshots*) dari hasil running kode Java `Main.java` pada Modul 7 yang membuktikan keberhasilan eksekusi seluruh skenario implementasi Abstract Class dan Interface:

1. Tangkapan Layar: Skenario 1 (Instansiasi Konkret) & Skenario 2 (Dynamic Binding Superclass Reference)



```
PowerShell — Modul 7 Skenario 1 & 2 Execution

PS C:\Users\dava1\...\CODE> java -cp "P7_3125522002_M Davana Yudista Firdausy" src.Main

=====
PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK - MODUL 7
ABSTRACT CLASS, ABSTRACT METHOD, AND INTERFACE
=====

Nama Mahasiswa : M. Davana Yudista Firdausy
NRP : 3125522002
Kelas : A D3 TI-SM
Institusi : PENS PSDKU Sumenep
Topik : Abstraksi & Kontrak Perilaku Sistem Inventaris Toko
=====

#####
### SCENARIO 1: OBJECT INSTANTIATION (CONCRETE SUBCLASSES) ###
#####

[Step 1.1] Menginstansiasi Objek Concrete Subclass Petugas:
-> [Superclass Constructor] Person dieksekusi untuk ID: PTG-001
-> [Subclass Constructor] Petugas dieksekusi untuk Jabatan: Supervisor Gudang

[Step 1.2] Menginstansiasi Objek Concrete Subclass Pemasok:
-> [Superclass Constructor] Person dieksekusi untuk ID: PMS-101
-> [Subclass Constructor] Pemasok dieksekusi untuk Perusahaan: PT Gramedia Mitra Sarana

[Step 1.3] Menginstansiasi Objek Concrete Class Barang:
-> [Concrete Instantiation] Objek Barang berhasil dibuat: Buku Tulis Hardcover

[Step 1.4] Menginstansiasi Objek Concrete Class Transaksi:
Berhasil [Barang]: Mengurangi 20 unit dari [BRG-001] Buku Tulis Hardcover (Sisa stok: 100)
Sukses [Transaksi TRX-OUT-2026-001]: Berhasil mencatat 20 unit [Buku Tulis Hardcover] ke dalam detail transaksi.
-> [Concrete Instantiation] Objek Transaksi berhasil dibuat: TRX-OUT-2026-001

[Analisis & Verifikasi Skenario 1]
[SUCCESS] Keempat objek concrete (Petugas, Pemasok, Barang, Transaksi) berhasil diinstansiasi.
[ENFORCEMENT] Perhatikan bahwa instruksi: 'Person p = new Person(...);' kini DITOLAK oleh compiler.
Error Compiler: 'Person is abstract; cannot be instantiated'.
Hal ini membuktikan Abstract Class berhasil mengamankan integritas domain model.

#####
### SCENARIO 2: ABSTRACT METHOD VIA SUPERCLASS REFERENCE ###
#####

[Step 2.1] Upcasting Petugas ke Referensi Person (Person p1 = new Petugas(...)):
-> [Superclass Constructor] Person dieksekusi untuk ID: PTG-002
-> [Subclass Constructor] Petugas dieksekusi untuk Jabatan: Kasir Senior

[Step 2.2] Upcasting Pemasok ke Referensi Person (Person p2 = new Pemasok(...)):
-> [Superclass Constructor] Person dieksekusi untuk ID: PMS-102
-> [Subclass Constructor] Pemasok dieksekusi untuk Perusahaan: CV Sinar Grafika

[Step 2.3] Memanggil Abstract Method prosesTugas() Melalui Referensi Person:
-> Pemanggilan via Referensi Person p1 (Objek Aktual: Petugas):
Hasil Eksekusi: [Petugas] Melayani pelanggan dan memverifikasi stok gudang.

-> Pemanggilan via Referensi Person p2 (Objek Aktual: Pemasok):
Hasil Eksekusi: [Pemasok] Menyiapkan manifes dan mendistribusikan barang ke gudang.

[Analisis & Verifikasi Skenario 2]
[DYNAMIC BINDING] Pada waktu kompilasi (compile-time), compiler hanya memeriksa bahwa tipe Person memiliki deklarasi method 'public abstract void prosesTugas()'.
Pada waktu eksekusi (runtime), JVM melalui Virtual Method Table (vtable) secara otomatis melompat ke implementasi spesifik milik Petugas dan Pemasok.
```

2. Tangkapan Layar: Skenario 3 (Realisasi Kontrak Interface) & Skenario 4 (Koleksi Polimorfik Heterogen)

```
PowerShell — Modul 7 Skenario 3 & 4 Execution

#####
### SCENARIO 3: INTERFACE METHOD VIA INTERFACE REFERENCE ###
#####

[Step 3.1] Polimorfisme Interface: DapatDiaudit d1 = new Barang(...):
-> Referensi DapatDiaudit d1 berhasil di-assign objek Barang (Spidol Whiteboard Hitam)

[Step 3.2] Polimorfisme Interface: DapatDiaudit d2 = new Transaksi(...):
Berhasil [Barang]: Menambah 40 unit ke [BRG-003] Kertas HVS A4 70gr (Total stok: 90)
Sukses [Transaksi TRX-IN-2026-002]: Berhasil mencatat 40 unit [Kertas HVS A4 70gr] ke dalam detail transaksi.
-> Referensi DapatDiaudit d2 berhasil di-assign objek Transaksi (TRX-IN-2026-002)

[Step 3.3] Eksekusi jalankanAudit() Melalui Referensi Interface DapatDiaudit:
1. d1.jalankanAudit() [Barang - Mendeteksi Stok Kritis]:
[AUDIT BARANG] Kode: BRG-004 | Nama: Spidol Whiteboard Hitam | Stok: 6 unit | Status: KRITIS (Stok Segera Habis - Restock Diperlukan) | Valuasi: Rp51.000,00

2. d2.jalankanAudit() [Transaksi - Rekap Aktivitas Mutasi]:
[AUDIT TRANSAKSI] ID: TRX-IN-2026-002 | Jenis: Barang Masuk | Total Diproses: 40 unit (1 SKU) | Total Nilai:
Rp1.920.000,00

[Analisis & Verifikasi Scenario 3]
[INTERFACE CONTRACT] Class Barang dan Transaksi TIDAK saling mewarisi (berbeda silsilah class).
Namun melalui Interface DapatDiaudit, keduanya tunduk pada satu kontrak perilaku.

#####
### SCENARIO 4: POLYMORPHIC COLLECTIONS (ARRAY / LIST) ###
#####

[Step 4.1] Pengujian Koleksi Polimorfik Abstract Class (Person[]):
Iterasi Koleksi Person[] (Ukuran: 4 elemen):
-----
[1] Tipe Runtime: Petugas | ID: PTG-001 | Nama: Ahmad Fauzi
Eksekusi prosesTugas(): [Petugas] Melayani pelanggan dan memverifikasi stok gudang.
-----
[2] Tipe Runtime: Petugas | ID: PTG-002 | Nama: Siti Nurhaliza
Eksekusi prosesTugas(): [Petugas] Melayani pelanggan dan memverifikasi stok gudang.
-----
[3] Tipe Runtime: Pemasok | ID: PMS-101 | Nama: Budi Santoso
Eksekusi prosesTugas(): [Pemasok] Menyiapkan manifes dan mendistribusikan barang ke gudang.
-----
[4] Tipe Runtime: Pemasok | ID: PMS-102 | Nama: Dewi Lestari
Eksekusi prosesTugas(): [Pemasok] Menyiapkan manifes dan mendistribusikan barang ke gudang.
-----

[Step 4.2] Pengujian Koleksi Polimorfik Interface (DapatDiaudit[]):
Iterasi Koleksi DapatDiaudit[] (Ukuran: 5 entitas terverifikasi):
[1] [AUDIT BARANG] Kode: BRG-001 | Nama: Buku Tulis Hardcover | Stok: 100 unit | Status: AMAN (Stok Terpenuhi) | Valuasi:
Rp1.500.000,00
[2] [AUDIT BARANG] Kode: BRG-004 | Nama: Spidol Whiteboard Hitam | Stok: 6 unit | Status: KRITIS (Stok Segera Habis -
Restock Diperlukan) | Valuasi: Rp51.000,00
[3] [AUDIT BARANG] Kode: BRG-003 | Nama: Kertas HVS A4 70gr | Stok: 90 unit | Status: AMAN (Stok Terpenuhi) | Valuasi:
Rp4.320.000,00
[4] [AUDIT TRANSAKSI] ID: TRX-OUT-2026-001 | Jenis: Barang Keluar | Total Diproses: 20 unit (1 SKU) | Total Nilai:
Rp300.000,00
[5] [AUDIT TRANSAKSI] ID: TRX-IN-2026-002 | Jenis: Barang Masuk | Total Diproses: 40 unit (1 SKU) | Total Nilai:
Rp1.920.000,00

[Analisis & Verifikasi Scenario 4]
[ABSTRACTION POWER] Dengan Polymorphic Collection, satu blok looping sederhana dapat
mengelola puluhan entitas berbeda secara elegan tanpa branching manual.
=====
SELURUH 4 SKENARIO MODUL 7 BERHASIL DIEKSEKUSI 100% DENGAN SUKSES!
=====
```